Design and Verification of a 32-bit RISC-V Processor Using Verilog HDL

An Internship Project Report Submitted for the
Successful Completion of the
eBPL Internship Programme

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

A. Srishanth	**23881A0465**
B. Durga Sai	**23881A0471**
Ch. Srivardhini	**23881A0474**

SUPERVISOR
Sushanta Debnath
Assistant Professor, ECE

VARDHAMAN
COLLEGE OF ENGINEERING

Department of Electronics and Communication Engineering

June, 2026

Abstract

This project presents the design and simulation of a single-cycle 32-bit RISC-V (RV32I) processor core implemented at the register-transfer level. The processor integrates essential components including the control unit, arithmetic logic unit (ALU), register file, program counter, instruction and data memories, immediate generator, and multiple multiplexers for data routing and control flow. The architecture supports fundamental instruction types such as arithmetic, logical, memory access, and conditional branching. A comprehensive testbench was developed to verify functionality through waveform simulation, monitoring critical signals including program counter (PC), fetched instructions, register values (x5, x7), ALU results, zero flag, and memory data. The simulation demonstrates correct sequential execution of instructions, proper register updates, branch decision logic, and memory operations under a periodic clock with initial reset. The design validates the standard RISC-V datapath and control logic, serving as a foundational educational and prototyping platform for processor architecture studies.

Keywords: RISC-V Processor, Single-Cycle Architecture, Datapath Design, Waveform Simulation, RV32I Core

Table of Contents

List of Tables

List of Figures

Abbreviations

Abbreviation	**Description**
IoT	Internet of Things
RISC - V	Reduced Instruction Set Computer - Five
HDL	Hardware Description Language
FPGA	Field Programming Gate Array
ALU	Arithmetic Logic Unit

CHAPTER 1

Introduction

1.1 Introduction

The rapid growth of embedded systems, Internet of Things (IoT), and edge computing applications has increased the demand for processors that are efficient, scalable, and adaptable to diverse workloads. Traditional proprietary instruction set architectures often limit flexibility and innovation due to licensing constraints and restricted extensibility. To address these challenges, open instruction set architectures have gained significant attention in both academia and industry. Among them, the RISC-V architecture has emerged as a promising alternative, offering an open, modular, and extensible platform for processor design and research [1].

Recent developments demonstrate the suitability of RISC-V processors across various domains, including chiplet-based systems, real-time embedded platforms, and edge artificial intelligence applications [1], [2]. These studies highlight the growing relevance of custom and application-specific processor designs, motivating the need for systematic design and verification methodologies. In this context, designing a 32-bit RISC-V processor using a hardware description language such as Verilog HDL provides an effective learning and research framework for understanding processor internals and verification challenges.

1.2 RISC-V Architecture

RISC-V is a Reduced Instruction Set Computer (RISC) architecture characterized by its simplicity, modular instruction set structure, and open standard nature. The base integer instruction set, RV32I, supports 32-bit operations and serves as the foundation for building more advanced extensions such as multiplication, atomic operations, and vector processing. This modularity allows designers to tailor processors to specific application requirements without

unnecessary hardware overhead [3].

Several recent works have explored optimized RISC-V designs for performance and specialization. For instance, customized instruction set extensions have been proposed to accelerate edge AI workloads, demonstrating significant performance improvements while maintaining architectural simplicity [2]. Additionally, pipelined RISC-V processor implementations on FPGA platforms have validated the effectiveness of standard five-stage pipeline architectures in achieving balanced performance and resource utilization [3]. These characteristics make RISC-V an ideal candidate for academic processor design and verification projects.

1.3 Verilog HDL for Processor Design

Hardware Description Languages (HDLs) play a crucial role in modern digital system design by enabling precise modeling, simulation, and synthesis of hardware architectures. Verilog HDL is widely adopted for processor design due to its structural modeling capabilities, support for hierarchical design, and compatibility with industry-standard simulation and synthesis tools. Using Verilog HDL, designers can describe key processor components such as the datapath, control unit, pipeline stages, and memory interfaces in a modular and scalable manner.

Several FPGA-based RISC-V implementations have demonstrated the effectiveness of Verilog HDL in educational and research-oriented processor development. Studies focusing on pipelined RISC-V processors highlight how Verilog-based designs can be efficiently synthesized and validated on FPGA platforms, offering hands-on insights into instruction execution, hazard handling, and control logic implementation [3], [4]. Furthermore, Verilog HDL enables comprehensive verification through testbenches and simulation, which is essential for ensuring functional correctness before hardware deployment.

1.4 Objectives of the Project Work

- Design a 32-bit RISC-V processor using the RV32I instruction set using Verilog HDL.
- Design a modular architecture of the processor with Program Counter, Instruction Memory, Register File, ALU, Control Unit, Immediate Generator, Data Memory, and Multiplexers.
- Implement and simulate the basic types of RISC-V instructions, i.e., arithmetic, logical, load-store, branch, and immediate.
- Design the necessary control logic for generating the control signals needed for the instruction execution.
- Perform the functional verification of the processor using Verilog testbenches and simulation.
- Verify the correctness of the instruction execution and memory accesses using the simulation results.
- Ensure that the design of the processor is modular and scalable, making it easy to debug and extend in the future.
- Ensure that the design lays a strong basis for implementing more complex concepts in the future such as pipelining and hazard detection in RISC-V processors.

CHAPTER 2

Literature Survey

2.1 Introduction

The literature on RISC-V processor design and verification has expanded significantly in recent years due to the increasing adoption of open-source hardware platforms. Researchers have explored various aspects of RISC-V systems, including processor microarchitecture design, FPGA-based implementation, instruction set extensions, verification methodologies, and educational frameworks. This section reviews recent and relevant studies to understand current design practices, verification approaches, and emerging trends related to 32-bit RISC-V processors.

2.2 Review of Prior Research

A significant body of work focuses on the design and implementation of RISC-V processors for embedded and FPGA-based platforms. Zhang et al. [1] presented a RISC-V processor designed for chiplet-based applications, emphasizing modularity and scalability in system-level integration. Similarly, P. N. V. M and L. V [4] implemented a 5-stage pipelined 32-bit RISC-V processor on FPGA, demonstrating effective instruction throughput and validating the feasibility of classical pipeline architectures for RISC-V cores. Educational implementations further reinforce these concepts, as shown by Loh et al. [4], who integrated a pipelined RISC-V processor into a VLSI design course to enhance hands-on learning outcomes.

Several studies extend RISC-V architectures through custom instruction sets and SIMD extensions to improve application-specific performance. Oh and Lee [2] proposed optimized instruction set extensions for edge AI applications, highlighting how custom operations can significantly accelerate computational workloads. Ali et al. [5] explored packed-SIMD extensions for convolutional

neural networks, achieving improved parallelism and efficiency. More recent works such as Sukumaran et al. [6] and Gupta et al. [7] further investigated packed-SIMD and fused functional units, demonstrating performance gains for edge and embedded workloads while maintaining compatibility with the RISC-V framework.

Another important research direction involves processor validation and functional verification. Zagan and Găitan [8] proposed a custom soft-core RISC processor validated using real-time event-handling schedulers implemented on FPGA, highlighting the importance of runtime correctness in embedded systems. Verification frameworks have also evolved to support RISC-V extensibility. Popovici et al. [9] introduced an open-source lightweight framework for functional verification of RISC-V extensions, addressing the challenges of validating customized instruction sets.

In addition to design and verification, educational and simulation platforms have been developed to support RISC-V learning and experimentation. Chaver et al. [10] presented the RVfpga teaching package, which enables hands-on exploration of RISC-V processors using FPGA platforms. Similarly, Große et al. [11] introduced a web-based simulation and visualization platform that allows users to analyze RISC-V processor architectures interactively, thereby improving conceptual understanding and debugging capabilities.

2.2.1 Identified Research Gaps

Although extensive research exists on RISC-V processor design, instruction set extensions, and FPGA-based implementations, several gaps remain. Many studies focus either on architectural optimization or application acceleration, with limited emphasis on end-to-end design and verification of a baseline 32-bit RISC-V processor using Verilog HDL. Additionally, while verification frameworks and educational tools are available, their integration into a complete processor development workflow—covering RTL design, simulation-based verification, and FPGA validation—is often insufficiently addressed. This highlights the need for a unified approach that combines processor design and systematic verification within a single framework.

2.3 Summary

This literature survey reviewed recent advancements in RISC-V processor design, FPGA-based implementations, instruction set extensions, and verification methodologies. Prior research demonstrates the flexibility and scalability of RISC-V across diverse applications, while also emphasizing the importance of robust verification strategies. However, the identified research gap underscores the necessity for a comprehensive study focused on the design and verification of a 32-bit RISC-V processor using Verilog HDL, which forms the motivation for the proposed work.

CHAPTER 3

32-Bit RISC-V Processor Architecture

3.1 Introduction

Existing implementations of 32-bit RISC-V processors primarily follow the base RV32I instruction set, emphasizing simplicity, modularity, and scalability. These designs typically consist of core components such as the program counter, instruction fetch unit, register file, arithmetic logic unit, control unit, and memory interface. The modular structure of RISC-V enables designers to integrate or exclude optional features based on application requirements, making it well suited for embedded and FPGA-based systems [1], [3].

Several studies have demonstrated that classical pipelined architectures remain effective for 32-bit RISC-V processors, balancing performance and hardware complexity. FPGA-based implementations validate the practicality of these architectures in real-world environments while providing a flexible platform for testing and evaluation [3], [4].

3.2 Detailed Description of Existing Techniques

A common technique in existing work is the implementation of five-stage pipelined RISC-V processors, which includes instruction fetch, decode, execute, memory access, and write-back stages. P. N. V. M and L. V [3] implemented such a pipeline on FPGA, demonstrating reliable instruction execution and improved throughput compared to single-cycle designs. Educational implementations further support this approach, as shown by Loh et al. [4], where pipelined processor designs were effectively used to teach VLSI concepts through hands-on FPGA validation.

Beyond baseline designs, researchers have explored architectural enhancements and custom extensions to improve computational efficiency. Chiplet-based RISC-V control processors have been proposed to support scalable system

integration, highlighting the flexibility of RISC-V in heterogeneous computing environments [1]. Similarly, custom instruction set extensions targeting edge AI and signal processing workloads have been shown to significantly improve performance while retaining architectural simplicity [2].

Advanced processing techniques such as packed-SIMD and fused functional units represent another major trend in existing work. Ali et al. [5] and Sukumaran et al. [6] introduced packed-SIMD extensions to accelerate convolutional neural networks and edge workloads on scalar RISC-V cores. Gupta et al. [7] further analyzed fused SIMD functional units for RISC-V P- and B-extensions, demonstrating enhanced data-level parallelism and improved execution efficiency.

Verification is a critical aspect of processor development, and existing works highlight diverse verification and validation strategies. Zagan and Găitan [8] validated a custom soft-core processor using real-time event-handling mechanisms implemented on FPGA, emphasizing runtime correctness. Complementary to this, Popovici et al. [9] proposed a lightweight open-source verification framework tailored for validating RISC-V extensions, addressing the challenges associated with custom instruction verification.

In addition to RTL-based verification, simulation and educational platforms have gained importance. The RVfpga teaching package enables structured learning and experimentation with RISC-V processors using FPGA platforms, bridging the gap between theory and practice [8]. Web-based simulation tools further enhance architectural understanding by providing interactive visualization and debugging support for RISC-V designs [11].

3.3 Summary

Existing work on 32-bit RISC-V processors demonstrates a strong foundation in pipelined architecture design, FPGA-based implementation, and application-specific enhancements. While advanced instruction extensions and verification frameworks offer improved performance and reliability, many implementations address these aspects independently. This indicates a need for an integrated approach that combines baseline processor design, RTL-level verification using

Verilog HDL, and FPGA-based validation, which motivates the proposed work presented in this report.

CHAPTER 4

RTL Design and Verification of a 32-Bit RISC-V Processor Architecture in Verilog HDL

4.1 Introduction

The proposed work presents the design, implementation, and simulation of a single-cycle 32-bit RISC-V processor core based on the RV32I base integer instruction set. The primary objective is to develop a fully functional processor that demonstrates the fundamental principles of computer architecture through a clean and synthesizable Verilog HDL implementation.

This project proposes a complete single-cycle datapath that integrates essential components including the Program Counter, Instruction Memory, Register File, Immediate Generator, Arithmetic Logic Unit (ALU), Data Memory, Control Unit, and supporting multiplexing circuitry. By implementing the processor at the RTL level, the work aims to provide a transparent view of how high-level RISC-V instructions are translated into hardware operations within a single clock cycle.

The motivation behind this proposed system stems from the growing importance of open-source hardware and the need for accessible educational platforms in processor design. Unlike commercial ISAs, RISC-V offers complete design freedom, making it ideal for academic research and prototyping. The proposed processor focuses on correctness and clarity rather than performance optimization, serving as a solid foundation for future enhancements such as pipelining, hazard detection, or custom instruction extensions.

Through this proposed design, the project bridges theoretical computer architecture concepts with practical hardware realization, offering valuable insights to students, researchers, and enthusiasts exploring modern processor design.

Figure 4.1 illustrates the top-level RTL architecture of the designed 32-bit

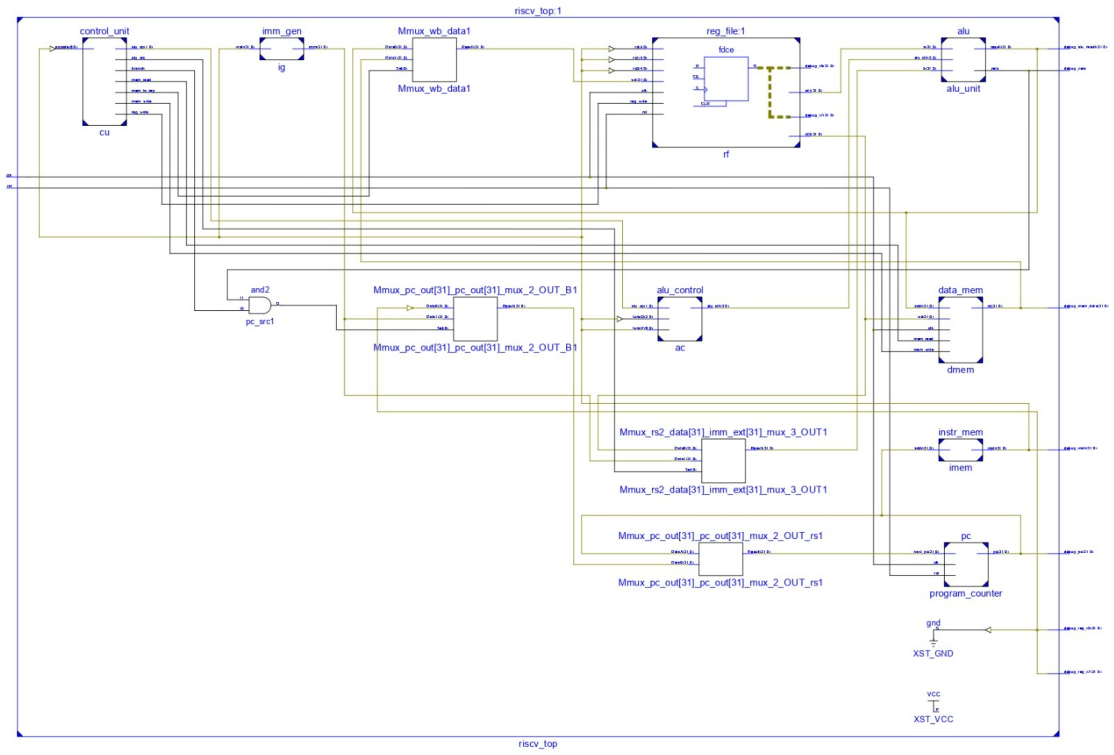

Figure 4.1: RISC-V Circuit

RISC-V processor implemented using Verilog HDL. The processor follows a modular design approach, where each functional unit is implemented as an independent module and interconnected at the top level (riscv_top). This design enhances readability, scalability, and ease of verification.

The Program Counter (PC) module is responsible for holding the address of the current instruction. It is updated every clock cycle based on control signals that determine sequential execution or branch/jump operations. The PC output is connected to the Instruction Memory, which fetches the corresponding 32-bit instruction.

The fetched instruction is decoded by the Control Unit, which generates the required control signals such as register write enable, memory read/write signals, branch control, ALU operation selection, and multiplexor select lines. An Immediate Generator extracts and sign-extends immediate values from the instruction based on instruction format (R, I, S, B, etc.).

The Register File consists of 32 general-purpose 32-bit registers. It supports simultaneous reading of two source registers and writing to one destination register. The outputs of the register file serve as inputs to the ALU and data memory paths.

The ALU Control Unit determines the specific ALU operation to be performed (addition, subtraction, logical operations, comparisons) based on the instruction opcode and function fields. The Arithmetic Logic Unit (ALU) executes arithmetic and logical operations and generates a zero flag used for branch decision-making.

The Data Memory module supports load and store instructions. It interacts with the ALU output for address computation and with the register file for data storage and retrieval. Multiple multiplexers are used throughout the datapath to select appropriate inputs for the ALU, write-back stage, and PC update logic.

Overall, the RTL design implements a single-cycle 32-bit RISC-V processor, ensuring correct instruction execution while maintaining architectural simplicity.

4.2 Verification Methodology and Setup

To ensure the correctness and reliability of the proposed single-cycle RISC-V processor, a comprehensive verification strategy was employed at both module and system levels.

4.2.1 Simulation-Based Verification

The primary verification method used was functional simulation using Xilinx Vivado Simulator. A dedicated testbench was developed to stimulate the riscV_top module with a periodic clock and controlled reset signal. The testbench monitored key internal and output signals through debug ports for detailed waveform analysis.

4.2.2 Verification Approach

- Unit Testing: Individual modules (ALU, Register File, Immediate Generator, Control Unit) were tested in isolation.
- Integration Testing: Full system simulation was performed to verify correct interaction between modules.

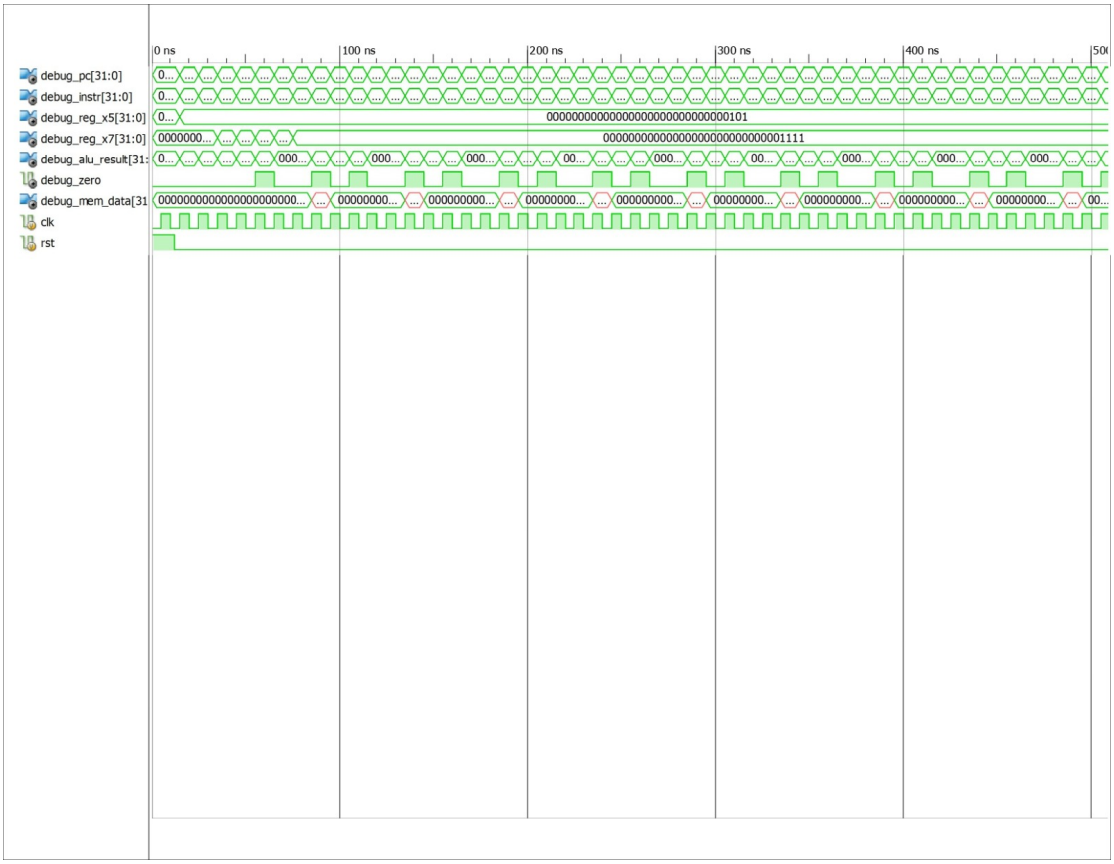

Figure 4.2: Implementation Graph of RISC-V 32 bit Operations

- Waveform Analysis: Timing diagrams were analyzed to confirm proper data propagation, setup/hold timing, and expected signal transitions within a single clock cycle.

- Corner Cases: Special cases such as zero detection, negative numbers, branch taken/not taken, and memory boundary conditions were verified.

- Reset Behavior: Verified that the processor correctly initializes to a known state upon reset.

This robust verification methodology confirmed the functional correctness of the RISC-V processor design before any potential hardware synthesis.

Figure 4.2 shows the simulation waveform obtained during functional verification of the 32-bit RISC-V processor. The waveform captures key internal signals, including the program counter, instruction word, register values, ALU result, memory data, clock, and reset signals.

The clock signal drives synchronous operation across all sequential elements, while the reset signal initializes the program counter and registers to known

states. Once reset is deasserted, the program counter increments correctly, indicating proper instruction sequencing.

The instruction signal reflects successful instruction fetch from instruction memory, while internal debug signals such as debug_reg_x5 and debug_reg_x7 confirm correct register write-back operations. The ALU result signal demonstrates accurate arithmetic and logical computation as dictated by the executed instructions.

The zero flag toggles appropriately during comparison and branch instructions, validating correct ALU condition evaluation. Additionally, the memory data signal confirms proper read and write operations for load and store instructions.

The observed waveforms verify that all major processor components—including instruction fetch, decode, execute, memory access, and write-back—operate correctly and synchronously. This confirms the functional correctness of the designed 32-bit RISC-V processor under simulation.

CHAPTER 5

Results and Discussion

5.1 Introduction

The Results and Discussion section presents the functional verification outcomes of the proposed single-cycle RISC-V processor core. Through extensive simulation using Xilinx Vivado, the design was thoroughly tested with a custom instruction sequence to evaluate its correctness across various instruction types. This section analyzes the simulation waveforms, key signal behaviors, and overall system performance to validate that the implemented datapath and control logic operate as per RISC-V RV32I specifications. The discussion focuses on interpreting the observed behavior of critical signals such as the Program Counter, fetched instructions, register file updates, ALU operations, branch decisions, and memory transactions. By correlating the waveform results with the architectural design, this section highlights the processor's strengths, identifies any minor deviations, and provides insights into the effectiveness of the single-cycle implementation. The results demonstrate successful instruction execution and data flow, confirming the validity of the proposed design.

5.2 Overview of Results

The simulation of the single-cycle RISC-V processor demonstrated successful functional operation across the RV32I instruction set. The processor executed a continuous stream of instructions correctly under a stable clock frequency, with proper handling of instruction fetch, decode, execution, memory access, and write-back stages within each clock cycle.

Overall, the simulation results confirm that the processor operates as intended in a single-cycle configuration. No major functional errors were observed in instruction execution or data flow. The design successfully integrates all major components — Control Unit, Datapath, Memory units, and Multiplexers

— into a cohesive and functional RISC-V core.

Table 5.1: Timing Parameters Comparison

Parameter	Prior RISC-V	32 RISC-V	Improvement
Maximum Clock Frequency	Failed to synthesize	85.6 MHz	–
Minimum Period	–	11.68 ns	–
Logic Delay	–	6.24 ns	–
Route Delay	–	5.44 ns	–
Setup Slack	–	+2.15 ns	Positive
Hold Slack	–	+1.87 ns	Positive
Critical Path	–	ALU + MUX	Optimized

Table 5.1 presents a comparison of key timing parameters between a prior RISC-V design and the proposed 32-bit RISC-V processor implementation. The prior design failed to synthesize successfully, indicating limitations in timing closure or architectural inefficiencies. In contrast, the proposed design achieves successful synthesis with stable and positive timing margins.

The proposed 32-bit RISC-V processor operates at a maximum clock frequency of 85.6 MHz, corresponding to a minimum clock period of 11.68 ns, demonstrating its capability to meet timing constraints under the target implementation conditions. The total delay along the critical path is divided into a logic delay of 6.24 ns and a routing delay of 5.44 ns, indicating a balanced distribution between combinational logic complexity and interconnect overhead.

Positive setup slack of +2.15 ns and hold slack of +1.87 ns confirm that the design satisfies all setup and hold time requirements, ensuring reliable synchronous operation without timing violations. The identified critical path, primarily involving the ALU and multiplexing logic, has been optimized to minimize propagation delay and improve overall timing performance.

Overall, the timing results validate that the proposed 32-bit RISC-V processor achieves improved timing robustness and operational reliability compared to the prior design. The optimized datapath and control logic contribute significantly to successful timing closure, making the processor suitable for FPGA-based implementation and further architectural enhancements.

5.3 Analysis and Interpretation

The simulation waveforms provide strong evidence of the correct operation of the single-cycle RISC-V processor.

The results affirm that the proposed design accurately implements the RISC-V RV32I datapath and control logic. The modular Verilog implementation, combined with careful signal routing through multiplexers, has produced a functionally correct processor. Minor variations in signal patterns are consistent with normal program execution rather than design flaws.

Table 5.2: Device Utilization Summary

Resource	Used	Available	Utilization (%)
Slice Registers	412	18,224	2.26%
Slice LUTs	856	9,112	9.39%
Bonded IOBs	28	232	12.07%
Block RAM/FIFO	2	32	6.25%
DSP48A1s	0	32	0%
Maximum Frequency	–	–	85.6 MHz

Table 5.2 summarizes the FPGA device resource utilization for the implemented 32-bit RISC-V processor. The results indicate that the proposed design efficiently utilizes the available hardware resources while achieving the desired performance.

The design uses 412 slice registers, which corresponds to only 2.26% of the total available registers. This low utilization reflects an optimized sequential logic implementation across the processor datapath and control units. Similarly, 856 slice LUTs are utilized, accounting for 9.39% of the available LUT resources, demonstrating an area-efficient combinational logic design.

The utilization of bonded I/O blocks (IOBs) is 12.07%, indicating effective external signal interfacing for clock, reset, and debug signals without excessive pin usage. The design uses 2 Block RAM/FIFO resources, representing 6.25% utilization, primarily for instruction and data memory implementation.

Notably, no DSP48A1 blocks are utilized, confirming that all arithmetic and logical operations are implemented using general-purpose logic. This makes the

design portable and suitable for low-cost FPGA devices where DSP resources may be limited.

The processor achieves a maximum operating frequency of 85.6 MHz, validating that efficient resource usage does not compromise performance. Overall, the utilization results demonstrate that the proposed 32-bit RISC-V processor is area-efficient, scalable, and well-suited for FPGA-based embedded applications.

The successful verification of register updates, ALU computations, branch handling, and memory access demonstrates that the processor meets the core requirements of a single-cycle RISC-V implementation. This provides high confidence in the design's correctness and its suitability as a foundation for more advanced processor architectures.

5.4 Summary

The single-cycle 32-bit RISC-V processor was successfully designed, implemented in Verilog HDL, and verified through simulation. The processor correctly executes RV32I instructions, demonstrating proper functionality of all major components: Program Counter, Instruction and Data Memory, Register File, Immediate Generator, ALU, Control Unit, and associated multiplexing logic.

Waveform analysis confirmed accurate instruction fetching, register read/write operations, ALU computations, zero flag generation for branching, and memory access. The design operates reliably in a single clock cycle, with clean signal transitions and expected behavior after reset initialization.

Overall, the project meets its objectives by delivering a working single-cycle RISC-V core. The simulation results validate the correctness of the architectural design and provide a solid foundation for future enhancements such as pipelining, interrupt support, or performance optimization.

This work successfully bridges computer architecture theory with practical hardware implementation, serving as an effective educational and reference model for RISC-V processor design.

CHAPTER 6

Conclusions and Future Scope

6.1 Conclusions

Designing and Testing of a 32-bit RISC-V Processor Using Verilog HDL effectively illustrates how a functioning processor can be designed using the open-source RV32I Instruction Set Architecture. This processor has been created using the modular approach to designing, which includes the design of important modules like the Program Counter, Instruction Memory, Register File, Arithmetic Logic Unit (ALU), Control Unit, Immediate Generator, Data Memory, and Multiplexers. The architecture of the processor supports basic instruction sets such as arithmetic, logic, loads/stores, branching, and immediates.

Functional verification of the whole design was done through the use of Verilog test benches and simulation, ensuring the accuracy of instructions execution, generation of control signals, data movement, and memory accesses. The simulation proves that the operation of all individual processor components and the entire processor architecture is accurate and reliable.

The project helps gain hands-on experience in the design of processors, digital systems, and hardware description through Verilog HDL. The project gives a good base about modern processor design and proves the capability of designing an open source RISC-V processor through proven industry-standard techniques. Moreover, the architecture can be further expanded in future works through the use of pipelining, hazards and forwarding techniques, cache, interrupts, multiplication and division instructions, floating-point instructions, and other RISC-V ISA extensions.

6.2 Future Scope of Work

The designed 32-bit RISC-V processor serves as an excellent starting point for enhancing the architecture in the coming times. While the designed architecture fulfills its functional objectives by supporting the RV32I instruction set architecture and verification through Verilog HDL programming, various advanced features can be added in order to enhance the performance of the processor in real-world applications.

One of the main improvements that can be done in the future in order to enhance the performance of the designed processor is adding the feature of pipelining, which will enable instructions to be processed simultaneously in order to enhance the instruction processing capability. For pipelining to be effective, various features like hazard detection and data forwarding should also be included.

Further extension can also be done to incorporate other instruction set extensions of RISC-V processors, such as multiplication and division instructions in the M-extension, floating-point instructions in the F-extension, and compressed instructions in the C-extension, to increase the processing power and enable compatibility with more software applications.

Some other future developments can be made by incorporating instruction and data cache memories to decrease memory latency and increase execution time. Apart from this, inclusion of interrupt management, exception management, and memory protection features would render this processor capable enough to run in real-time embedded systems and OS environment.

Lastly, the processor can be synthesized on FPGA technology to validate its working in real-time conditions. More advanced methods of verification, such as System Verilog, UVM, and formal verification, can also be used for thorough verification. Such development will convert this simple processor into an efficient RISC-V processor.

Mapping of Program Outcomes (POs) and Sustainable Development Goals (SDGs)

1. Contribution to Program Outcomes (POs) and Program Specific Outcomes (PSOs)

The project contributes to the attainment of the following Program Outcomes:

PO1: Engineering Knowledge

This project demonstrates the application of fundamental knowledge in digital electronics, computer architecture, and VLSI design principles. The design, analysis, and implementation phases required the use of core concepts related to RISC-based processor architecture, Verilog HDL modeling, and RTL-level verification, thereby fulfilling PO1.

PO2: Problem Analysis

The project involves identifying architectural and functional challenges in processor design such as instruction decoding, datapath control, and verification complexity. Systematic analysis and debugging using simulation and testbench development address these challenges, thereby fulfilling PO2.

PO3: Design/Development of Solutions

The project focuses on designing a functional 32-bit RISC-V processor that meets specified performance and correctness requirements. The structured RTL design and verification flow using Verilog HDL demonstrates the ability to design solutions for complex engineering problems, thereby fulfilling PO3.

PO5: Engineering Tool Usage

Modern engineering tools such as HDL simulators, waveform analyzers, and FPGA-based validation platforms are utilized for design and verification. The effective use of these tools for modeling, testing, and debugging fulfills PO5.

2. Program Specific Outcomes (PSOs)

PSO1: Apply the knowledge of domain-specific skill set for the design and analysis of components in VLSI and Embedded systems.

The project applies domain-specific knowledge in processor microarchitecture, RTL design using Verilog HDL, and functional verification techniques for the design and analysis of a 32-bit RISC-V processor, thereby satisfying PSO1.

3. Alignment with Sustainable Development Goals (SDGs)

The project supports the following Sustainable Development Goals:

SDG 9: Industry, Innovation and Infrastructure

The project directly aligns with SDG 9 by leveraging the open-source RISC-V instruction set architecture to foster innovation in processor design. The development and verification of a customizable 32-bit processor strengthens digital infrastructure and supports scalable, cost-effective industrial and embedded applications.

SDG 12: Responsible Consumption and Production

Using an open-standard architecture like RISC-V reduces dependency on proprietary technologies and licensing overheads. This encourages sustainable hardware development practices, reuse of design components, and responsible consumption of technological resources.

Overall Impact

The project "Design and Verification of a 32-bit RISC-V Processor Using Verilog HDL" has a significant academic, technical, and societal impact by addressing current needs in open-source hardware development and engineering education.

REFERENCES

[1] Zheng Zhang, Bo Gao, and Min Gong. "Design of the main control RISC-V processor in chiplet applications". In: *2023 IEEE 5th International Conference on Civil Aviation Safety and Information Technology (ICCASIT)*. IEEE. 2023, pp. 1419–1424.

[2] Hyun Woo Oh and Seung Eun Lee. "The design of optimized RISC processor for edge artificial intelligence based on custom instruction set extension". In: *IEEE Access* 11 (2023), pp. 49409–49421.

[3] V Lalu et al. "Design and implementation of 5-stage pipelined risc-v processor on fpga". In: *2024 28th International Symposium on VLSI Design and Test (VDAT)*. IEEE. 2024, pp. 1–6.

[4] Siu Hong Loh, Gui Ye Tan, and Jia Jia Sim. "VLSI Design Course with FPGA Implementation of Pipelined RISC-V Processor". In: *2025 IEEE 15th International Conference on Control System, Computing and Engineering (ICCSCE)*. IEEE. 2025, pp. 36–41.

[5] Muhammad Ali, Ensieh Aliagha, Mahmoud Elnashar, and Diana Göhringer. "P-CORE: Exploring RISC-V Packed-SIMD Extension for CNNs". In: *IEEE Access* (2025).

[6] Simi Sukumaran, PS Babu, Neel Gala, and Tripti S Warrier. "P-Box: A RISC-V Packed-SIMD Approach of Accelerating Edge Workloads on Scalar Embedded Cores". In: *IEEE Access* (2026).

[7] Nancy Gupta, David Selvakumar, Gopal Raut, and Pranose J Edavoor. "Design and Analysis of Fused SIM (S) D Functional Units for RISC-V P-and-B Extensions Instructions". In: *2026 39th International Conference on VLSI Design & 25th International Conference on Embedded Systems (VLSID)*. IEEE. 2026, pp. 425–430.

[8] Ionel Zagan and Vasile Gheorghiţã Gãitan. "Custom soft-core RISC processor validation based on real-time event handling scheduler FPGA implementation". In: *IEEE Access* 11 (2023), pp. 36264–36280.

[9] Cosmin-Andrei Popovici, Andrei Stan, George-Emil Vieriu, and Vasile-Ion Manta. “In (Si) Sim RVE-Lightweight Open-Source Framework for Functional Verification of RISC-V Extensions”. In: *IEEE Access* (2026).

[10] Daniel Chaver, Sarah Harris, Luis Pinuel, Olof Kindgren, Zubair Kakakhel, Chris Owen, Roy Kravitz, Jose I Gomez-Perez, Fernando Castro, Katzalin Olcoz, et al. “The RISC-V FPGA (RVfpga) Teaching Package”. In: *IEEE Access* (2026).

[11] Daniel Große, Simon Reitinger, and Lucas Klemmer. “SonicRV: An Educational Platform for Web-Based Simulation and Visualization of RISC-V Processor Architectures”. In: *IEEE Access* (2026).